Mini Project I: "TrustVerify" – A CLI Tool for File Integrity

Team Members: **Abdurrakhman Kalumian [220208955]**

&

Muhammet İbrahimoğlu [230208992]

Course: SENG 473 (1) Information Security.

Instructor: MUHAMMET MUSTAFA ÖLMEZ

Date: 29 March 2026

Introduction

In modern digital communications, ensuring that a file has not been tampered with and verifying the true identity of its sender are critical security requirements. In this project, we developed a Python-based Command Line Interface (CLI) tool called TrustVerify. The main purpose of this tool is to ensure both the integrity and authenticity of files exchanged between a sender and a receiver.

The system implements a robust two-layer security architecture:

- **SHA-256 Hashing:** Used to create a unique fingerprint for files to ensure local data integrity.
- **RSA Digital Signatures**: Used to verify the identity of the sender and ensure non-repudiation through asymmetric cryptography.

System Architecture and Methodology

The system is designed to operate in two distinct phases to counter different types of security threats, such as Data Poisoning and Man-in-the-Middle (MitM) attacks.

Phase 1: Hashing and Local Integrity

In this phase, the tool focuses on detecting unauthorized modifications to local files.

1. **Hash Generation:** The system reads any given file (text, PDF, or image) in binary mode and generates a unique SHA-256 hash.
2. **Manifest Creation:** A metadata.json file is generated, acting as a secure directory manifest that stores the filenames and their corresponding SHA-256 hashes.
3. **Integrity Checking:** The tool can scan the directory and compare the current hashes of the files against the ones stored in metadata.json. If a single byte is altered, the new hash will drastically change, and the tool will flag the file as modified.

Phase 2: Digital Signatures and Authenticity

Integrity alone is insufficient for secure file exchange. Phase 2 introduces RSA cryptography.

1. **Key Generation:** The cryptography library is used to generate a secure 2048-bit Public/Private RSA key pair for the sender.
2. **Signing Process:** The tool hashes the metadata.json file and encrypts this hash using the sender's Private Key. This process outputs a digital signature (signature.sig).
3. **Verification Process:** The receiver uses the file, the metadata.json, the signature, and the sender's Public Key. The tool decrypts the signature using the Public Key and compares it to the hash of the received metadata.json. A match confirms the file's origin and integrity.

Implementation Details

The project code is modular, well-structured, and strictly utilizes hashlib and cryptography libraries as required. It consists of the following modules:

- hash.py: Contains the generate_hash(file_path) function, which safely opens files in rb mode and returns the SHA-256 hex digest.
- local_integrity.py: Manages the directory scanning. It includes create_manifest() to build the JSON dictionary and check_integrity() to compare current file states against the old manifest.
- sign.py: Handles all RSA operations. It features generate_keys() with PEM serialization, sign_manifest() using PSS padding and SHA256 for optimal security, and verify_signature() to validate incoming data.
- main.py: Acts as the central entry point for the CLI. It uses the sys.argv module to parse user commands (e.g., generate_keys, manifest, check, sign, verify) and execute the corresponding functions smoothly.

Security Analysis

Q1: Why Hashing Alone Is Not Enough to Prove Identity?

Hashing provides **integrity** but entirely lacks **authenticity** or identity proof. A hash function (e.g., SHA-256) is completely public; anyone can compute a hash for any file. If an attacker intercepts the communication, modifies a file (like text.txt), and regenerates a new metadata.json with the updated hashes, the check_integrity() function would falsely report that everything is "OK". This happens because the tool is comparing the files against the attacker's tampered manifest. Therefore, while hashing proves a file hasn't changed since the hash was recorded, it cannot answer *who* created the file or the manifest.

Q2: How Does the RSA Public/Private Key Relationship Ensure Non-Repudiation?

Non-repudiation ensures that a sender cannot falsely deny having sent a specific message or file. This is mathematically guaranteed by the asymmetric nature of RSA cryptography:

- **Exclusive Control:** The Private Key is strictly kept secret by the sender. No one else has access to it.
- **The Signing Mechanism:** When the sender uses their Private Key to encrypt the hash of metadata.json, they create a unique digital signature tied specifically to both the document and their private identity.
- **Undeniable Proof:** This signature can *only* be successfully decrypted and verified using the sender's exact Public Key. If the verification passes, it provides definitive, mathematically sound proof that the signature could only have been generated by the owner of that Private Key. The sender cannot claim that someone else forged the signature, hence ensuring non-repudiation.

Conclusion

The TrustVerify project effectively illustrates the importance of integrating multiple cryptographic concepts. By using SHA-256, we can efficiently detect any data tampering at the bit level. Moreover, by implementing RSA digital signatures, we protect the process against advanced identity spoofing and man-in-the-middle (MitM) attacks. This tool provides a robust solution for secure, verifiable, and indisputable file transfers.